

FATIH SULTAN MEHMET VAKIF UNIVERSITESI

Software Verification, Validation & Security Testing

FORMAL TEST PLAN

Smart Hospital Appointment System

Team Name: ZeroDay Squad

Project Title: TestHub

Date: April 2026

Team Members:

Saed O. S. Radi — [REDACTED]

Anas I. A. Alhatti — [REDACTED]

Abdulrahman Aljabahji — [REDACTED]

Ahmed Alsaleh — [REDACTED]

1. Project and Application Overview

The Smart Hospital Appointment System is a full-stack web application that manages hospital operations, patient appointments, and medical records. It connects three user roles — Patients, Doctors, and Administrators — in a unified digital environment.

Technology Stack:

- Backend: Java Spring Boot 3.x with Spring Security and Role-Based Access Control (RBAC)
- Frontend: React.js for dynamic dashboards, Thymeleaf for secure server-side authentication pages
- Database: MySQL 9.1 relational database
- Security: HTTPS with self-signed SSL certificate, HttpOnly session cookies (JSESSIONID)

Key Features:

- Patient registration, login, and appointment booking with department and specialist selection
- Doctor dashboard for viewing schedules, accepting/rejecting appointments, and writing digital prescriptions using the OpenFDA API
- Admin control panel for creating, editing, and deleting Doctors and Patients, and managing hospital departments

2. Testing Scope

2.1 In Scope

The following features and functionalities are included in this testing effort:

Authentication & Session Management:

- Login for all three roles: Admin, Doctor, and Patient
- Patient self-registration
- Logout and session termination
- Session persistence after browser page refresh
- Role-Based Access Control (RBAC) enforcement across all dashboards

Patient Module:

- Registration with valid and invalid data — weak password, duplicate username, empty fields
- Patient login and dashboard access
- Browsing hospital departments and viewing available doctors
- Appointment booking flow
- Viewing medical history and prescription details
- Boundary value testing: minimum password length (8 characters), minimum username length (4 characters)

Doctor Module:

- Doctor login and dashboard access
- Viewing assigned appointments
- Accepting and rejecting appointment requests
- Writing and saving prescriptions using OpenFDA medication search
- Using the Dashboard Refresh button

Admin Module:

- Admin login and dashboard access
- Creating, editing, and deleting doctor accounts — including empty field validation
- Creating, editing, and deleting patient accounts — including empty field validation
- Managing hospital departments

Form Validation:

- Empty field validation on all forms: Login, Registration, Create Doctor, Create Patient, Edit Doctor, Edit Patient, Write Prescription, and Book Appointment
- Invalid data testing: weak passwords, duplicate usernames, invalid characters in username, username too short
- Boundary value analysis on password (exact 8 chars) and username (exact 4 chars)

Security Testing:

- SQL Injection on login form
- Cross-Site Scripting (XSS) on registration form
- CSRF protection verification
- Brute force protection on login endpoint
- Session cookie security (HttpOnly and Secure flags)
- Directory traversal attack prevention
- Sensitive data exposure in URL parameters
- Unauthorized page access by wrong roles
- Cross-patient data leakage via admin edit form
- OWASP ZAP automated security scan

Performance Testing:

- Response time under 10, 50, and 100 concurrent users
- Login endpoint under 20 concurrent POST requests

- Patient Dashboard load time under 10 concurrent users
- Booking endpoint under 20 concurrent users
- Admin Dashboard under 15 concurrent users

Usability Testing:

- Task-based evaluation with 5 independent testers using Think-Aloud Protocol
- Visual design and color consistency evaluation
- Typography and readability assessment
- Layout and navigation consistency across all pages
- Error message clarity and actionability rating
- Responsive feedback and loading indicators verification
- Overall user satisfaction scoring (target: $\geq 3.5/5$)

Automated Testing (Selenium WebDriver):

Selenium WebDriver will be used as an automation tool to execute all functional test cases programmatically. The automated tests will cover all 58 functional test scenarios including authentication, registration, patient module, doctor module, and admin module — they will not represent separate test cases but will automate execution of existing functional test cases.

2.2 Out of Scope

The following features, scenarios, and testing types are explicitly excluded from this testing effort. Any questions about these areas fall outside the agreed testing boundaries:

Functional Features Not Implemented:

- Email notification system — no email functionality exists in the current version
- Password reset / Forgot Password feature — not implemented in the application
- Patient self-service profile editing — only Admin can edit patient profiles
- Doctor availability calendar — no calendar or availability management feature exists
- Appointment cancellation by patient — no cancellation feature implemented
- Search functionality for doctors, patients, or appointments
- Pagination of long user or appointment lists
- File upload — no support for medical images or documents
- Multi-language or internationalization support
- Print prescription or export to PDF feature
- Patient-to-doctor messaging or chat
- Appointment reminders or push notifications
- Payment or billing functionality

Form Validation Not Tested:

- Session timeout after extended idle period — requires waiting 30+ minutes
- Concurrent same-user login from multiple browsers simultaneously
- Admin deleting their own account — risk of system lockout

Security Areas Not Tested:

- IDOR (Insecure Direct Object Reference) — patient accessing another patient's records via URL manipulation
- API-level RBAC bypass using Postman or similar tools
- Session fixation attacks
- Clickjacking protection testing
- Man-in-the-Middle (MITM) attack simulation
- SSL/TLS certificate validation beyond browser warning acceptance
- Advanced penetration testing beyond OWASP ZAP automated scan

Performance Areas Not Tested:

- Database query optimization under extreme load (10,000+ records)
- Memory leak testing over extended runtime periods
- CDN caching and static asset performance
- Long-running session performance degradation

Usability Areas Not Tested:

- Mobile device responsiveness — application is desktop web-only
- Accessibility compliance (WCAG 2.1) for users with disabilities
- Cross-browser testing — only Google Chrome will be used
- Screen reader compatibility
- Dark mode or theme customization

Infrastructure Not Tested:

- Cloud deployment (AWS, Azure, or any production environment)
- Database backup and recovery procedures
- Load balancing configuration
- CI/CD pipeline integration testing
- Docker containerization or orchestration
- Third-party OpenFDA API reliability or rate limiting

3. Testing Types and Techniques

The following testing approaches will be applied throughout this project:

Black Box Testing:

Will be applied in all Functional, Usability, and Performance test cases. Test cases will be designed based on system requirements and expected behaviour without any knowledge of internal code structure. Testers will interact only with the UI and API endpoints, treating the system as a black box.

White Box Testing:

Will be applied in Security test cases that require direct inspection of source code and configuration files, such as verifying security settings in SecurityConfig.java and reviewing backend controller logic for potential vulnerabilities.

Gray Box Testing:

Will be applied in Security testing using tools such as OWASP ZAP and Chrome DevTools, where partial knowledge of the application architecture (Spring Security, session management) will be used to conduct targeted tests. Also applicable in Selenium automation where UI element structure is known in advance, and JDBC is used to inject test data directly into the database before test execution.

Testing Type	Techniques Used	Tools
Functional Testing	Black-box testing, boundary value analysis, equivalence partitioning, negative testing, form validation, RBAC verification. Selenium WebDriver used to automate execution of all 58 functional test cases. JDBC used for test data setup and cleanup.	Selenium WebDriver 4.18.1, WebDriverManager 5.7.0, Manual Testing
Performance Testing	Load testing, stress testing, response time measurement under concurrent users	Apache JMeter 5.6.3
Usability Testing	Task-based evaluation, Think-Aloud Protocol, structured observation, satisfaction rating scale (1–5)	Manual observation with 5 independent testers
Security Testing	SQL injection, XSS, CSRF, brute force simulation, session analysis, directory traversal, RBAC bypass attempts	OWASP ZAP 2.17, Chrome DevTools, IntelliJ IDEA, Manual testing

4. Roles and Responsibilities

Role	Member	Responsibilities
Test Manager	Saed O. S. Radi	Leads the team, coordinates planning, manages schedule and milestones, reviews all test documents, ensures all deliverables are completed on time, and oversees overall testing strategy
Test Engineer	Anas I. A. Alhatti	Designs all test cases, selects testing techniques, executes functional tests manually, records results, and documents defects with steps to reproduce
Programmer	Abdulrahman Aljabahji	Develops and maintains Selenium automation scripts, configures OWASP ZAP, sets up Apache JMeter, executes security and performance tests, and documents results
Interface Designer / Usability Tester	Ahmed Alsaleh	Designs and leads usability test sessions using Think-Aloud Protocol, coordinates 5 independent testers, analyses qualitative feedback, and documents usability findings with severity ratings

5. Test Schedule

Week(s)	Phase	Key Activities & Deliverables
Week 3	Kick-off & Team Setup	Formed group, assigned roles, chose team name 'ZeroDay Squad', registered with instructor
Week 4	Application Selection & Scope	Selected Smart Hospital Appointment System as test target, defined testing scope, began drafting Test Plan
Weeks 5–6	Test Planning	Complete Formal Test Plan. DELIVERABLE: Submit Test Plan
Weeks 7–8	Test Case Design	Designed 88 test cases across all four testing types. DELIVERABLE: Submit Test Case Design Document
Weeks 9–11	Environment Setup & Dry Run	Set up test environment: Apache JMeter, OWASP ZAP, Selenium WebDriver. Conducted dry runs of key test cases to verify environment readiness
Weeks 12–14	Test Execution	Execute all 88 test cases. Record actual results, log defects, collect screenshots as evidence. Conduct usability tests with 5 independent testers using Think-Aloud Protocol
Week 15	Analysis & Reporting	Analyse test results. Finalise Test Execution Report and Defect Report. Prepare final presentation slides
Week 16	Final Submission & Presentation	FINAL DELIVERABLE: Submitted all four documents. Delivered 30-minute group presentation to instructor

6. Entry and Exit Criteria

6.1 Entry Criteria

- Smart Hospital Appointment System is fully deployed and accessible — backend on <https://localhost:8080>, frontend on <http://localhost:3000>
- All user accounts verified in the database: Admin (testadmin / Admin@1234), Doctor (doctor2 / Doctor@1234), Patient (patient1 / Patient@1234)
- MySQL 9.1 database running on localhost:3306 with hospital_db schema populated
- All required testing tools installed and configured: Apache JMeter 5.6.3, OWASP ZAP 2.17, Selenium WebDriver 4.18.1, WebDriverManager 5.7.0
- All 88 test cases designed, reviewed, and approved
- 5 independent usability testers available and briefed on Think-Aloud Protocol

6.2 Exit Criteria

- At least 95% of planned test cases must be executed before testing is considered complete
- All Critical and High severity defects must be identified, documented, and resolution attempted
- Performance test results must be collected for all 7 JMeter scenarios
- OWASP ZAP automated security scan must be completed and findings documented
- Usability evaluation must be completed with all 5 independent testers
- All four deliverable documents must be complete and submitted

7. Tools and Environments

7.1 Test Environment

- Operating System: macOS (Apple Silicon)
- Browser: Google Chrome (version 147)
- Backend: Java Spring Boot 3.x running on https://localhost:8080 (HTTPS with self-signed SSL)
- Frontend: React.js running on http://localhost:3000
- Database: MySQL 9.1 running on localhost:3306, schema: hospital_db
- IDE: IntelliJ IDEA 2025.1
- Java Version: JDK 23

7.2 Testing Tools

- Selenium WebDriver 4.18.1 with WebDriverManager 5.7.0 — automated execution of all 58 functional test cases via Chrome browser control. JDBC used for test data setup and cleanup via direct MySQL connection.
- Apache JMeter 5.6.3 — performance and load testing with thread groups and Summary Report listener
- OWASP ZAP 2.17 — automated security vulnerability scanning (Spider + Active Scan)
- Google Chrome DevTools — cookie inspection, network analysis, session monitoring
- MySQL Workbench — direct database inspection and SQL queries for test data verification
- IntelliJ IDEA — Selenium script development, execution, and console output review

8. Risk Identification and Mitigation

Risk	Severity	Mitigation Strategy
Test environment instability — server crashes or database connection failures during execution	High	Document all setup steps; maintain database backups; keep restart procedures documented and accessible
Core features non-functional due to bugs (e.g. appointment booking)	High	Document bugs immediately as defects; adapt test execution to test available features; use direct MySQL insertion for dependent test scenarios
Insufficient time to execute all 88 test cases	High	Prioritise Critical and High severity test cases first; document any skipped tests with justification in Out of Scope section
OWASP ZAP or JMeter tool configuration issues	Medium	Pre-configure all tools before test execution phase begins; use default scan profiles as fallback; document configuration steps
SSL certificate warnings preventing browser access on localhost	Medium	Use Chrome advanced settings to bypass self-signed certificate warnings; configure Selenium with --ignore-certificate-errors flag
Usability testers unfamiliar with Think-Aloud Protocol	Medium	Brief all 5 testers in advance; prepare simple instruction cards explaining the Think-Aloud method before sessions begin
Missing test evidence (screenshots not captured)	Low	Take screenshots immediately after each test execution; organise by test case ID in a dedicated screenshots folder in the project repository
Circular dependency issues in Spring Security configuration	Medium	Isolate Spring beans into separate configuration classes to prevent circular references; test each configuration change independently